

Universidad de Nueva Extremadura

Facultad de Ingeniería

Análisis

Material No. 6

Ingeniería de Software II

Ing. Mauricio Moo Aguilar

1.

GIT y GitHub: Control de Versiones para el Desarrollo Colaborativo

En el desarrollo de software moderno, el trabajo en equipo, el control de versiones y la colaboración efectiva son fundamentales. Aquí es donde entran en juego herramientas como *GIT* y *GitHub*. Ambas han revolucionado la forma en que los programadores trabajan, permitiendo un manejo eficiente del código fuente, facilitando el trabajo colaborativo y evitando errores catastróficos al modificar el software.

¿Qué es GIT?

GIT es un sistema de control de versiones distribuido creado por *Linus Torvalds*, el mismo creador del núcleo de *Linux*. Un sistema de control de versiones permite a los desarrolladores llevar un registro de los cambios realizados en los archivos del proyecto, facilitando el trabajo colaborativo y la reversión de errores si es necesario.

Características principales de GIT

- **Distribuido**: Cada desarrollador tiene una copia completa del repositorio con todo su historial.
- **Eficiente**: GIT es rápido, especialmente para proyectos grandes.
- **Seguro**: Cada cambio se almacena de forma única y segura mediante algoritmos hash.
- **Flexible**: Admite flujos de trabajo variados (centralizados, colaborativos, jerárquicos).

Supongamos que estás trabajando en una aplicación web con un equipo. **GIT** te permite crear una *rama (branch)* para trabajar en una nueva funcionalidad sin afectar la versión principal (*main* o *master*). Si algo sale mal, puedes volver a un punto anterior con un simple comando.

¿Qué es GitHub?

GitHub es una plataforma en la nube que permite almacenar repositorios *GIT*, facilitando el trabajo colaborativo entre desarrolladores desde cualquier parte del mundo. Ofrece una interfaz gráfica amigable, herramientas de colaboración y características avanzadas como *pull requests*, *issues*, y *actions*.

GitHub no es el único servicio de alojamiento de repositorios (también existen *GitLab* y *Bitbucket*), pero es el más popular y utilizado en la industria y la academia.

Principales características de GitHub:

- **Repositorios Remotos**: Puedes subir tu código para que esté disponible en línea.
- **Colaboración**: Varias personas pueden contribuir a un mismo proyecto.
- **Control de Cambios**: Puedes revisar la historia de cambios, quién hizo qué y cuándo.
- **Documentación**: Puedes documentar tu proyecto con archivos como *README.md*.
- **Integración Continua**: *GitHub Actions* permite automatizar pruebas y despliegues.

Relación entre GIT y GitHub

- *GIT* es la herramienta de control de versiones, que puedes usar localmente.
- *GitHub* es una plataforma basada en la web que aloja proyectos GIT y permite trabajar en equipo de forma remota.

En resumen: *GIT* funciona sin conexión, *GitHub* requiere estar en línea. Usar *GitHub* sin conocer *GIT* no tiene sentido, ya que *GitHub* es solo la interfaz visual de un repositorio *GIT*.

Flujo básico de trabajo con GIT y GitHub

1. Crear un repositorio local con *GIT*:

   git init

2. Agregar archivos al repositorio:

   git add .

3. Guardar los cambios con un *Commit*:

   git commit -m "Primer commit"

4. Conectar con un repositorio remoto en *GitHub*:

   git remote add origin https://github.com/usuario/repositorio.git

5. Subir los cambios al repositorio remoto:

   git push -u origin main

6. Actualizar tu copia con los cambios del equipo:

   git pull origin main

Conceptos básicos a dominar

- **Repositorio**: Carpeta donde se guarda el código y el historial de cambios.
- **Commit**: Registro de un conjunto de cambios.
- **Branch** (rama): Línea de desarrollo paralela.
- **Merge**: Fusión de ramas.
- **Clone**: Copiar un repositorio remoto a tu computadora.
- **Push**: Enviar cambios locales al repositorio remoto.
- **Pull**: Traer cambios del repositorio remoto al local.

Buenas prácticas en el uso de GIT y GitHub

- Realiza *commits* frecuentes con mensajes claros.
- Usa ramas para nuevas funcionalidades o correcciones.
- No subas archivos innecesarios (usa *.gitignore*).
- Documenta tu código y proyecto.
- Revisa el código antes de hacer *merge* (revisión por pares).
- Usa los *issues* para registrar errores o tareas pendientes.

Conclusiones

Aprender a usar *GIT* y *GitHub* es fundamental para cualquier profesional de *TI*. No solo son herramientas indispensables en el mundo laboral, sino que también promueven una cultura de colaboración, responsabilidad y mejora continua.

Fuentes Bibliográficas:

Brais Moure – (2023) Git y GitHub desde cero 2da Edición, Publicación Independiente.